Module 3: Scheduling and Synchronization

Exploring CPU scheduling algorithms, device scheduling mechanisms, process synchronization techniques, and deadlock management in operating systems

CPU Scheduling

Algorithms, implementations, and real-world scheduling strategies

Device Scheduling

I/O operations, disk scheduling algorithms, and system implementation

Process Synchronization

Critical sections, mutexes, semaphores, and monitor mechanisms

Deadlocks

Detection, prevention, avoidance, and recovery strategies

Based on Operating Systems: Three Easy Pieces

Date: January 14, 2026

Introduction to Scheduling and Synchronization

Operating systems must efficiently manage concurrent execution, resource allocation, and process coordination to ensure system stability and performance in complex computing environments.

CPU Scheduling

Determines the order in which processes access the CPU, balancing efficiency and responsiveness

Device Scheduling

Manages access to I/O devices, minimizing seek time and maximizing throughput

Process Synchronization

Ensures safe concurrent access to shared resources, preventing race conditions and data inconsistency

Why It Matters

These mechanisms are essential for understanding how operating systems efficiently and reliably manage multiple processes, shared resources, and system stability in complex computing environments.

CPU Scheduling Overview

What is CPU Scheduling?

CPU scheduling is fundamental to maximizing system efficiency by determining the order in which processes access the CPU.

It complements device scheduling by ensuring efficient use of the central processing unit when processes are ready to execute.

Goal

To optimize system performance by balancing metrics like throughput, response time, and CPU utilization.

Performance Metrics

- Throughput: Processes per unit time
- Response time: Time to first response
- Waiting time: Time in ready queue
- Turnaround time: Complete execution time

Scheduling Criteria

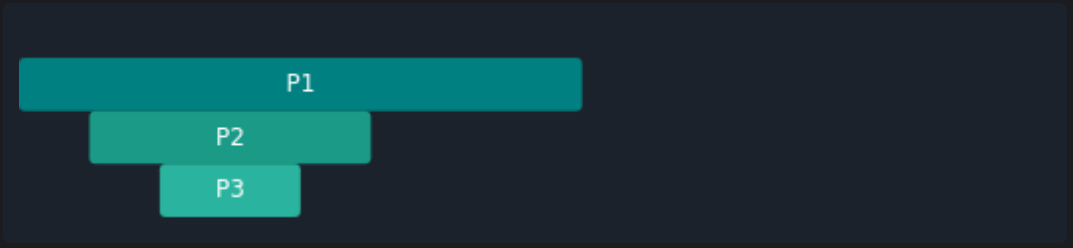
- Maximize CPU utilization
- Maximize throughput
- Minimize waiting time
- Minimize response time

Algorithm Types

FCFS | SJF | RR | Priority

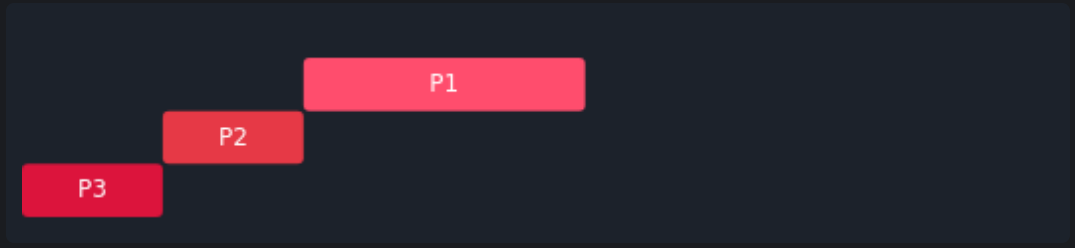
Basic CPU Scheduling Algorithms

First-Come, First-Served (FCFS)



- **Character:** Non-preemptive algorithm that executes processes in arrival order
- **Advantage:** Simple implementation and fair ordering based on arrival time
- **Limitation:** Suffers from convoy effect, leading to high average waiting times
- **Example:** If a long process arrives first, all subsequent short processes must wait, resulting in poor average waiting time performance

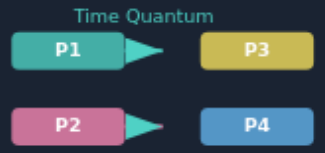
Shortest Job First (SJF)



- **Character:** Algorithm that prioritizes shorter processes to minimize average waiting time
- **Advantage:** Improves responsiveness by allowing shorter processes to preempt longer ones (SRTF variant)
- **Limitation:** Requires accurate burst time prediction, which is often impractical in real systems
- **Note:** SJF is optimal for minimizing average waiting time but needs to know the burst time before execution

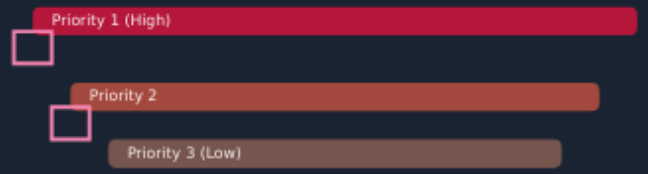
Advanced CPU Scheduling Algorithms

Round Robin (RR)



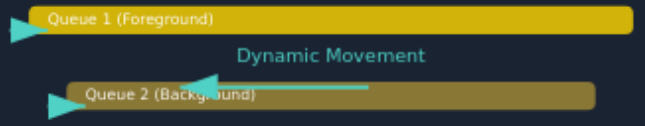
- Uses time quantum to ensure fair CPU allocation
- Each process executes for a fixed time slice before being preempted
- Prevents starvation and ensures all processes get regular CPU time
- Performance depends on quantum size:
  - Too small: increases context switch overhead
  - Too large: reduces responsiveness

Priority Scheduling



- Assigns execution order based on process priority
- Higher-priority processes execute first
- Risks starvation for low-priority processes
- Mitigated by **aging**:
  - Priority increases the longer a process waits in ready queue
  - Ensures all processes eventually execute

Multilevel Queue (MLQ)



- Classifies processes into queues with different scheduling policies
- **Fixed queues** (e.g., foreground vs. background)
- **Multilevel Feedback Queue (MLFQ)** allows:
  - Dynamic movement between queues based on behavior
  - Adaptive behavior:
    - Interactive processes promoted to higher-priority queues
    - CPU-bound processes demoted

Real-World CPU Scheduling Implementation

Modern operating systems combine multiple scheduling approaches to balance fairness, responsiveness, and efficiency.

Linux Implementation

- Multilevel feedback queue with O(1) scheduling
- Time slices inversely proportional to priority
- Adaptive behavior based on process activity

Windows Implementation

- 32-level priority scheme
- Time quantum adjustments based on process type
- Foreground/background status consideration

Hybrid Approaches

Both systems balance fairness, responsiveness, and efficiency through adaptive scheduling mechanisms that dynamically adjust to workload patterns.

Device Scheduling Fundamentals

Device scheduling manages access to I/O devices, minimizing seek time and maximizing throughput. It complements CPU scheduling by ensuring efficient use of peripheral resources.

Purpose & Goals

- Minimize seek time and rotational latency
- Maximize system throughput
- Ensure fair access to devices
- Reduce waiting time for I/O requests

Key Components

- I/O request queues
- Device drivers
- Scheduling algorithms
- Completion handlers

Disk Scheduling Impact

Disk scheduling plays a critical role in system performance by reducing mechanical delays in accessing data. Effective disk scheduling can significantly improve response time and overall system throughput, especially in systems with high I/O loads.

Disk Scheduling Algorithms

FCFS (First-Come, First-Served)

Schedules requests in arrival order. Simple but inefficient with high average seek times.

SSTF (Shortest Seek Time First)

Prioritizes closest requests. Reduces response time but risks starvation for distant requests.

SCAN (Elevator Algorithm)

Moves head in one direction, servicing all requests until end of disk, then reverses.

C-SCAN (Circular SCAN)

Treats disk as circular list. Returns to beginning without servicing on return trip.

Performance Comparison

Algorithm	Average Seek Time	Starvation Risk	Fairness	Use Case
FCFS	High	Low	High	Simple systems
SSTF	Low	High	Low	Performance-critical
SCAN	Medium	Low	Medium	General-purpose
C-SCAN	Medium	Low	High	Uniform response

Starvation Prevention Strategies

Aging
Increases priority of waiting requests

Fair Scheduling
Ensures equal service distribution

Modern Device Scheduling Systems

Modern operating systems implement sophisticated I/O scheduling algorithms that dynamically adapt to workload patterns, balancing throughput, fairness, and response time.

Deadline Scheduler

- Prioritizes requests based on deadlines to ensure timely completion
- Ideal for real-time applications with strict timing requirements
- Minimizes latency for time-sensitive operations

CFQ (Completely Fair Queuing)

- Aims to provide fair CPU and I/O time to all processes
- Particularly beneficial in multi-user environments
- Enhances system responsiveness for interactive workloads

Adaptive Behavior

These schedulers dynamically adapt to workload patterns, providing optimal performance characteristics for different use cases while maintaining system fairness and responsiveness.

Process Synchronization Concepts

Critical Section

Code segment accessing/modifying shared resources

- **Mutual Exclusion**: Only one process can execute in critical section at a time
- **Progress**: Process wishing to enter should be able to do so without unnecessary delay
- **Bounded Waiting**: Limit on number of times other processes can enter after a request

Race Condition

Occurs when multiple processes access shared data simultaneously

- Final outcome depends on relative timing of execution
- Difficult to detect due to timing dependencies
- Primary motivation for synchronization mechanisms

Critical Section Visualization

Synchronization Requirements

Ensuring safe concurrent access to shared resources

- Prevent data corruption and inconsistent states
- Coordinate access to shared data
- Form foundation for advanced mechanisms (mutexes, semaphores, monitors)

Synchronization Mechanisms

Synchronization mechanisms ensure exclusive access to shared resources and coordinate process execution to prevent race conditions and data inconsistency.

Mutex

Mutual exclusion lock ensuring exclusive access to a shared resource

- Only one process/thread can hold the lock at a time
- Supports acquire and release operations
- Implemented using atomic hardware instructions
- Used to protect critical sections

Common in Linux and Windows kernels

Semaphore

Synchronization object maintaining a count to control access to resources

- Supports wait() and signal() operations
- Binary semaphore: values 0 or 1
- Counting semaphore: non-negative integer values
- Used in producer-consumer problems

Manages multiple instances of resources

Monitor

High-level construct encapsulating shared data and procedures

- Automatic mutual exclusion
- Includes condition variables
- Supports wait() and signal() operations
- Solves classical synchronization problems

Used for readers-writers and dining philosophers problems

These mechanisms are critical for building reliable, concurrent applications in both system software and user-level programs.

Real-World Synchronization Implementation

Modern operating systems implement advanced synchronization mechanisms to ensure reliable, concurrent execution in both system software and user-level programs.

Linux

Adaptive Mutexes
Spinlock on single-core, sleep on multi-core systems

Reader-Writer Locks
Optimizes for read-heavy workloads

Condition Variables
For thread coordination

Windows

Critical Sections
Optimized for low-contention scenarios

Events
For thread synchronization

Semaphores
Through Windows API

Programming Languages

Java
Built-in synchronization via synchronized blocks and Monitor classes

C#
Monitor classes for synchronization

POSIX Threads (pthreads)
Mutexes, condition variables, and barriers

These mechanisms are critical for building reliable, concurrent applications in both system software and user-level programs.

Deadlock Fundamentals

Deadlocks occur when processes are blocked indefinitely, each waiting for a resource held by another. Understanding the conditions that lead to deadlocks is essential for prevention and detection.

Mutual Exclusion

At least one resource must be held in a non-shareable mode, preventing concurrent access by multiple processes.

Hold and Wait

A process holding at least one resource must wait while requesting additional resources held by other processes.

No Preemption

Resources cannot be forcibly removed from processes; they must be released voluntarily.

Circular Wait

A set of processes forms a circular chain where each process is waiting for a resource held by the next process in the chain.

Deadlock Visualization

Deadlock Detection and Prevention

Deadlock Detection

Resource Allocation Graph (RAG)

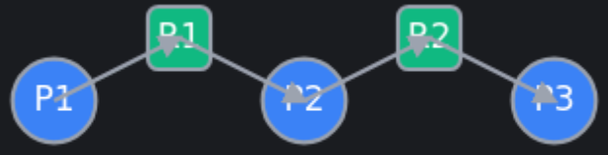
Graph-based algorithm to identify deadlocks by detecting cycles in the resource allocation system.

- If a cycle exists in a RAG with single-instance resources, a deadlock is confirmed.
- Effective for systems with one resource instance per type.

Wait-For Graph

Simplified version of RAG, showing only process-to-process relationships.

- Directs cycle detection in the system.
- Reduces complexity compared to full RAG.



Deadlock Prevention

Coffman Conditions

Four necessary conditions for deadlock:

- Mutual Exclusion
- Hold and Wait
- No Preemption
- Circular Wait

Prevention Strategies

- Global Resource Ordering: Assign numbers to resources, requiring processes to request resources in ascending order.
- Request All Resources: Process must request all resources at once or not at all.
- Prevent Circular Wait: Break the circular wait condition by ensuring resources are allocated in a way that prevents cycles.

Eliminating any one Coffman condition prevents deadlock, but may reduce resource utilization.

Deadlock Avoidance and Recovery

Banker's Algorithm

A dynamic deadlock avoidance algorithm that checks if a resource allocation leads to a safe state.

Key Requirements

- Processes must declare maximum needs in advance
- System simulates execution to ensure safety
- Only allocates resources if result is safe

```
1. Request = [allocation[i]] + [need[i]]
2. If request ≤ available, allocate resources
3. Simulate execution to find safe sequence
4. If no safe sequence exists, abort process
```

Recovery Strategies

Process Termination

Abort one or more deadlocked processes

Benefits: Simple to implement

Drawbacks: Loss of progress, resource waste

Resource Preemption

Take resources from a victim process and roll back

Benefits: Avoids complete process termination

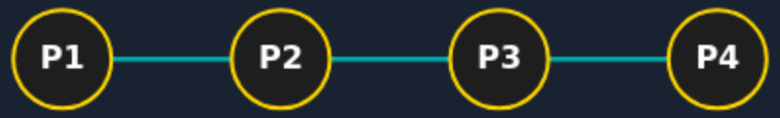
Drawbacks: Complexity, potential data inconsistency

Recovery strategies are critical for maintaining system availability in the event of deadlocks.

Real-World Deadlock Management

Wait-For Graph Detection

Database systems use wait-for graphs to identify deadlocks by detecting cycles in the resource allocation graph.



Cycle detected: Deadlock exists

Deadlock Detection

- Periodically scan resource allocation graph for cycles
- Identify all deadlocked processes
- Abort one or more processes to break cycles
- Rollback aborted transactions to a safe state

Transaction Rollback Strategies

- **Least Costly Transaction:** Abort the transaction with the minimum cost (e.g., least work done)
- **First Request:** Abort the process that made the first request in the deadlock cycle
- **Timeout-based:** Abort transactions that have been waiting too long

Database Implementation

PostgreSQL

Uses wait-for graphs with cycle detection. Aborts the transaction with the fewest max locks.

MySQL

Implements deadlock detection with timeout-based rollback strategies.

Key Benefits

Minimal disruption to system operation, automatic deadlock resolution, and efficient resource utilization.

Integration and Performance Considerations

Modern operating systems must balance scheduling efficiency, synchronization safety, and deadlock prevention to achieve optimal performance while maintaining system reliability.

Scheduling Efficiency

- Maximizes throughput
- Ensures fairness
- Optimizes response time

Synchronization Safety

- Prevents data corruption
- Maintains consistency
- Ensures atomicity

Deadlock Prevention

- Balances safety and performance
- Minimizes disruption
- Ensures system stability

Integration Challenges

- Coordination between scheduling and synchronization can impact performance
- Deadlock prevention mechanisms must not undermine efficiency

Effective operating systems balance these mechanisms to enable reliable, efficient multitasking in complex computing environments.

Conclusion and Future Directions

Scheduling and synchronization are foundational to OS design, enabling reliable, efficient multitasking in modern computing environments.

CPU Scheduling

Maximizes throughput and fairness while balancing system responsiveness and efficiency.

Device Synchronization

Minimizes seek time and maximizes throughput in I/O operations.

Process Synchronization

Ensures data integrity when multiple processes access shared resources.

Future Directions

- Adaptive scheduling algorithms that learn from workload patterns
- New approaches to deadlock prevention in multi-core systems
- Hardware-assisted synchronization mechanisms
- Integration of machine learning in scheduling decisions

"The foundation of reliable computing."